Fine-tuning a <=3B LLM for Text2SQL

A SQL-generation assistant for fintech data access

Author: Muhammad Iqbal Hilmy Izzulhaq Date: June 2026

1. Summary

Non-technical teams in fintech often need data that is locked behind SQL and the Data Science team's queue. This project fine-tunes a **small (<=3B) open-source LLM** to translate natural-language questions into **executable SQL**, so a chatbot can answer questions like "Who were the top performing merchants in the last quarter?" directly.

I fine-tune **Qwen2.5-Coder-1.5B-Instruct** (1.54B params, Apache-2.0) with **QLoRA** using **Unsloth**, on the **BIRD** benchmark (which ships real SQLite databases so accuracy can be measured by executing the queries). The full pipeline - exploration -> preprocessing -> training -> inference -> execution-based evaluation - is implemented and the non-GPU portion is verified end-to-end. The GPU fine-tuning step is delivered as a Colab-ready notebook (free T4).

> ***What was executed for this report.** The data, prompting and execution-evaluation code was run locally and verified end-to-end on a synthetic fintech database (Section 7). The QLoRA training step is designed for and runs on a free Colab T4 (Section 6); the results table (Section 8) is structured to be filled with the numbers from that run. This division is deliberate and called out honestly rather than hidden.*

2. Problem & objective

Input: a database schema + a user's natural-language question (optionally a hint / external knowledge). **Output:** a single executable SQL query that answers the question.

Constraints from the brief:

- Model size **<= 3B**; smaller is better.
- Free, open-source datasets and tooling (Google Colab is acceptable compute).
- Accuracy is secondary to demonstrating a sound, complete methodology.
- Executable queries are rewarded.

Success criteria I optimized for: (1) the model emits syntactically valid SQL almost always, and (2) as often as possible the query is executable and returns the correct result (execution accuracy).

3. Tools & libraries (all free / open-source)

Layer	Choice	Reasoning
Base model	Qwen2.5-Coder-1.5B-Instruct	Code-specialised (pretrained on code incl. SQL), Apache-2.0, 1.54B <= 3B, fits a free T4. The 0.5B variant (494M, Apache-2.0) is used for the "smaller is better" ablation; the 3B variant is license:other, so 1.5B is the best permissive trade-off.
Fine-tuning	QLoRA (4-bit base + LoRA adapters)	Trains a 1.5B model on a single 16 GB T4. Only ~0.5-1% of params are trainable, so it's fast and the adapter is a few MB.

Layer	Choice	Reasoning
Speed/memory	Unsloth	~2x faster, ~50% less VRAM than vanilla PEFT via fused kernels; ships pre-quantized model mirrors. Crucial for free-tier compute.
Trainer	TRL SFTTrainer	Standard supervised fine-tuning; supports completion-only loss masking.
Data	HF datasets, BIRD, SynSQL-2.5M	BIRD provides real DBs for execution eval; SynSQL adds cross-domain scale (2.5M pairs, 16k+ DBs, Apache-2.0).
Evaluation	sqlite3 (stdlib)	Execution accuracy needs nothing more than running both queries and comparing result sets.
Compute	Google Colab (T4)	Free GPU; the training notebook is Colab-native.

Why a code model, not a general one? SQL is code. A coder base already knows SQL syntax, common idioms (JOIN/GROUP BY/window functions) and is robust to schema formatting in the prompt - so LoRA only has to teach the task framing (schema -> question -> query), not the language itself. This is the single most important lever at the <=3B scale.

---

4. Dataset exploration & analysis

4.1 BIRD (primary)

BIRD (BIg Bench for LaRge-scale Database grounded text-to-SQL) is a cross-domain benchmark of question->SQL pairs over **95 real databases** spanning 37 domains (~12.7k pairs total; ~9.4k train / ~1.5k dev). Distinguishing features that drove my design:

- **Real, messy databases shipped as SQLite** -> I can measure execution accuracy, not just string match. This is why BIRD is my primary set.
- **"Evidence"** - each question carries an external-knowledge hint (e.g. a formula or a code->label mapping). Feeding it into the prompt is worth several EX points and mirrors how a real assistant would be given business context.
- **Difficulty labels** (simple / moderate / challenging) on dev -> lets me report per-bucket accuracy and set expectations (challenging = nested, multi-join).
- **Large, wide schemas** -> the schema dominates the prompt. This motivated the schema-serialization design (Section 5.2) and max_seq_length = 2048.

notebooks/01_data_exploration.ipynb quantifies: difficulty distribution, SQL length (tokens), keyword frequency (JOIN / GROUP BY / nested SELECT / aggregation), and schema breadth (tables & columns per DB). The observed pattern - **JOIN + GROUP BY + aggregation dominate** - is exactly the shape of the motivating fintech question, which is reassuring for transfer.

4.2 SynSQL-2.5M (augmentation)

The first million-scale, fully-synthetic cross-domain Text2SQL dataset: **2.5M samples over 16,000+ databases**, Apache-2.0 (the data behind the OmniSQL models). Each record is self-contained (it ships the schema as DDL text, so no .sqlite files are needed for training). I use a **subset** (the brief says the full set isn't required) to test whether broader schema/domain coverage improves generalization (Exp 3). Its scale is its strength and its risk: being synthetic, its distribution differs from BIRD's human questions, so I treat it as augmentation, not a replacement.

4.3 Implication for the recipe

1. Schema is the bulk of every prompt -> serialize it compactly and keep context long enough to avoid truncating large schemas.
2. Always include BIRD's evidence/hint.
3. Train the model to produce JOIN/GROUP BY/aggregation reliably - the common, high-value patterns.
4. Expect most gains on simple/moderate; report challenging separately.

---

5. Methodology

5.1 Prompt format (identical at train & inference)

A train/serve prompt mismatch silently destroys accuracy, so a single function (src/prompts.py) builds the chat messages used everywhere:

```
system : You are an expert data analyst who writes correct, executable SQLite SQL ...
user   : D
```

(The assistant target is wrapped in a fenced sql block at training time; the model learns to emit that exact format.) Training computes loss **only on the assistant turn** (completion-only masking), so the model is graded on the SQL it must produce, not on reproducing the schema. At inference the SQL is parsed back out of the fenced block by extract_sql.

5.2 Schema serialization

Schemas are reconstructed **directly from the SQLite file** so they always match the database the query is later executed against. Two modes:

- ddl (default): the original CREATE TABLE statements - most informative (types, keys, FKs).
- compact: table(col TYPE, ...) one line per table - cheaper on tokens for very wide schemas.

5.3 Fine-tuning configuration (Exp 1)

Hyper-parameter	Value	Note
Base	Qwen2.5-Coder-1.5B-Instruct	4-bit (QLoRA)
LoRA rank /	16 / 16	adapters on all attention + MLP projections
LoRA dropout	0.0	Unsloth-optimized (no recompute)
Max seq length	2048	fits most BIRD schemas
Epochs	2	(subsample 8k for time-boxed Colab)
LR / schedule	2e-4 / cosine, 3% warmup	standard QLoRA
Batch (eff.)	8 x 2 = 16	grad accumulation
Optimizer	adamw_8bit	memory-light
Precision	bf16 if supported else fp16	-

All of this is one preset in src/config.py; experiments override single fields.

6. Experiment design & reasoning

Each experiment isolates one variable so the result is interpretable.

1. **Baseline (no fine-tuning).** Same base model, zero-shot, same prompt.

Why: attributes any gain specifically to fine-tuning rather than to the base model's prior or to prompt engineering. Without it, "we got X% EX" is meaningless.

2. **Exp 1 - main run: 1.5B + BIRD + QLoRA.** The core deliverable.

Why: establishes the headline number and validates the whole pipeline on the primary benchmark.

3. **Exp 2 - smaller model (0.5B), same recipe.**

Why: the brief explicitly rewards smaller models. This quantifies the accuracy<->size trade-off and tells us whether a 0.5B model is "good enough" to deploy cheaply, or whether 1.5B is needed.

4. **Exp 3 - data scaling: BIRD + SynSQL subset.**

Why: tests the hypothesis that broader cross-domain coverage improves generalization to unseen schemas (the realistic deployment condition). Run for 1 epoch because the combined set is larger.

Further ablations enabled by the config (low-cost, high-information): with vs. without the evidence hint; ddl vs. compact schema; LoRA rank 8/16/32. These are deliberately cheap to run and each answers a specific design question.

7. Pipeline & what was verified

The pipeline is four composable stages, each a small CLI module:

```
data_prep.py   BIRD/SynSQL  -> standardized JSONL (schema + prompt messages + gold)
```

```
train.py
```

Verified end-to-end (CPU, no GPU, no downloads). scripts/smoke_test.py builds a synthetic fintech database (merchants + transactions), runs it through data_prep, and checks the evaluation logic. Real console output:

```
2) preprocess (data_prep.load_bird)
```

```
  [PASS] l
```

This proves the preprocessing and the execution-accuracy metric are correct (gold->100%, broken->penalized) before a single GPU-hour is spent - the part most likely to contain silent bugs. The motivating question resolves to a sensible top-merchants query:

```
SELECT m.name, SUM(t.amount) AS total
```

```
FROM merch
```

8. Results

***How to populate this table:** run notebooks/02_finetune_qlora.ipynb (train) then notebooks/03_inference_eval.ipynb (eval). The eval notebook writes outputs/metrics_*.json and a before_after.png chart, and the run produces the screenshots the brief asks for (training loss curve, evaluation summary).*

Run	Execution acc. (EX)	Valid-SQL rate	Exact match
Baseline (1.5B, zero-shot)	fill	fill	fill
Exp 1 (1.5B + BIRD, QLoRA)	fill	fill	fill
Exp 2 (0.5B + BIRD, QLoRA)	fill	fill	fill
Exp 3 (1.5B + BIRD + SynSQL)	fill	fill	fill

Reference expectations (from the literature, to calibrate - not my results). BIRD dev is hard: at release GPT-4 scored ~46% EX and the human ceiling is ~92%. A time-boxed QLoRA fine-tune of a 1.5B model should be expected to land in the **~15-30% EX** range with a **high valid-SQL rate (>80%)** - i.e. it reliably writes runnable SQL, and gets the right answer on a meaningful minority, concentrated in the simple/moderate buckets. Fine-tuning should clearly beat the zero-shot baseline (which often emits non-executable or schema-hallucinating SQL). These are the numbers to sanity-check your run against.

Insert here: training loss curve (Colab), evaluation summary screenshot, and the baseline-vs-fine-tuned bar chart (report/figures/before_after.png).

9. Weaknesses & suggestions for improvement

Methodology weaknesses (honest assessment):

- **No schema linking.** The entire schema is dumped into the prompt. On wide

BIRD databases the relevant columns get lost, and large schemas risk truncation. -> Add a retrieval step that selects the likely-relevant tables/ columns before prompting (e.g. embed columns + question, keep top-k). This is usually the single biggest lever for small models.

- **Greedy, single-sample decoding.** No self-correction. -> Self-consistency

(sample N, pick the majority result set) and **execution-guided decoding** (run candidates, keep one that executes and returns non-empty) directly raise EX and valid-SQL - and the execution harness for this is already in src/evaluate.py.

- **Small model, small data, few epochs.** Capacity- and data-limited by design.

-> Scale the SynSQL subset; consider the 3B model if its license fits; tune LoRA rank.

- **Value/literal errors.** The model can't see actual cell values, so it guesses

string literals and date formats. -> Inject a few **sample rows per table** (supported via with_samples=True) or a value-retrieval step.

- **Evaluation caveats.** EX with result-set comparison can over-credit

(different query, same rows by coincidence) or under-credit (column ordering, ties). Per-difficulty reporting partly mitigates this; BIRD's official evaluator and the **Soft-F1 / VES** metrics would harden it.

- **Single seed, no validation-driven early stopping.** -> Multiple seeds + a held-

out check for variance and over-fitting.

Productionization (beyond accuracy):

- **Safety:** never execute generated SQL with write access. Run **read-only**,

in a sandboxed replica, with a statement-timeout (already enforced in the evaluator) and row limits. Reject anything that isn't a single SELECT.

- **Guardrails:** validate generated SQL against the schema (table/column exists) and re-prompt on failure.
- **Latency/cost:** the QLoRA adapter is tiny; serve the 4-bit base + adapter, or the 0.5B model if Exp 2 shows it's adequate.

10. References

- BIRD-bench - https://bird-bench.github.io
- SynSQL-2.5M / OmniSQL - https://huggingface.co/datasets/seeklhy/SynSQL-2.5M (arXiv:2503.02240)
- Qwen2.5-Coder - https://huggingface.co/Qwen/Qwen2.5-Coder-1.5B-Instruct (arXiv:2409.12186)
- Unsloth - https://github.com/unslothai/unsloth
- TRL (SFTTrainer) - https://huggingface.co/docs/trl
- PEFT / QLoRA - Dettmers et al., 2023, QLoRA: Efficient Finetuning of Quantized LLMs (arXiv:2305.14314)